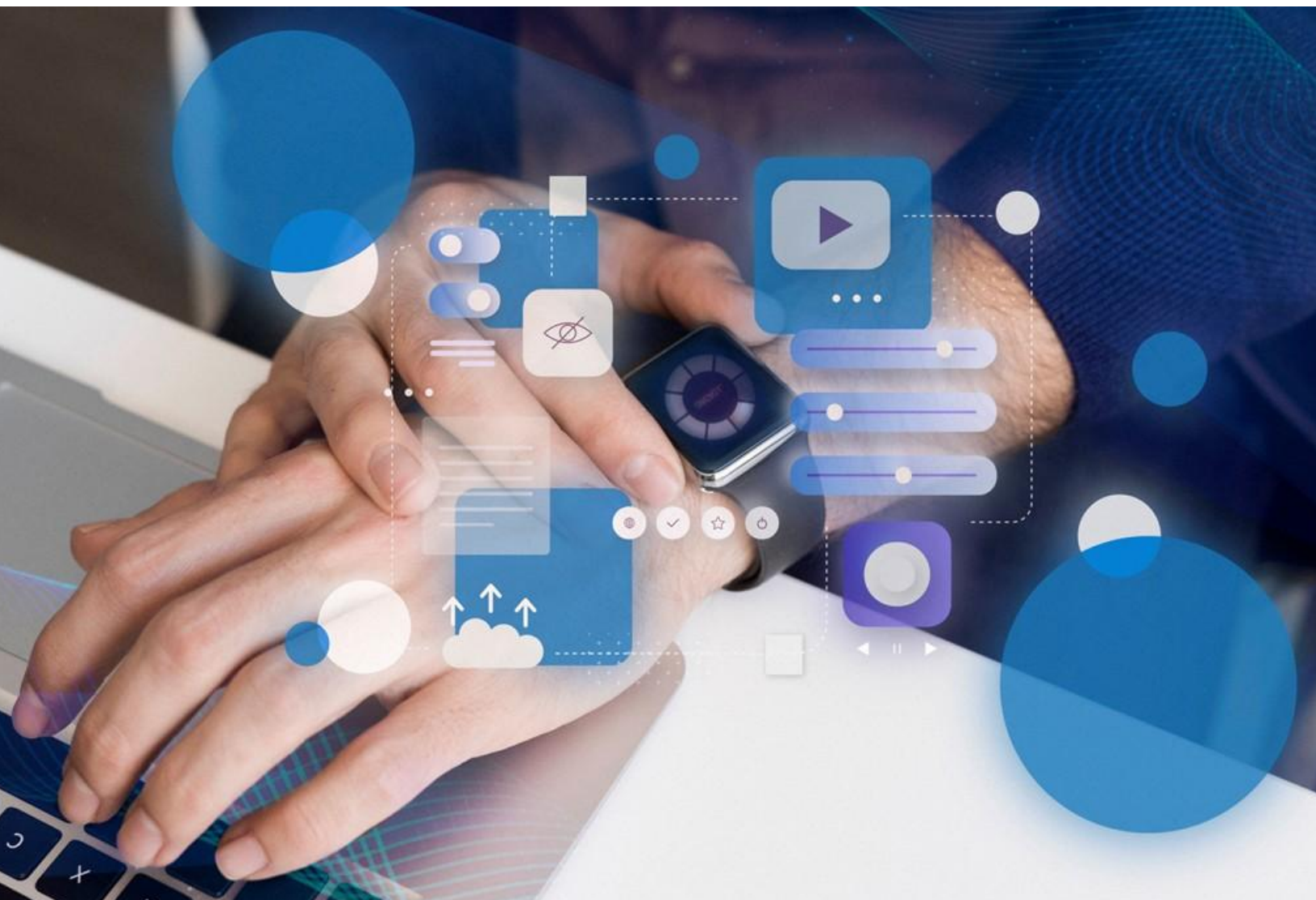


# Taller de plataformas web

Introducción a los sistemas de bases de datos



## **Escuela de ingeniería, energía y tecnología**

**Director:** Marcelo Lucero

### **Elaboración**

**Experto disciplinar:** Jaime Gómez

**Diseñadora instruccional:** Paula Lopera

**Editora instruccional:** Emilia De la Cruz

### **Validación**

**Experto disciplinar:** Jaime Gómez

**Jefa de Producción Tecnopedagógica:** Sandra Betancur

### **Equipo de desarrollo**

Welearn

**AÑO**

2026

# Tabla de contenidos

|                                                                                                        |           |
|--------------------------------------------------------------------------------------------------------|-----------|
| <i>Aprendizaje esperado de la semana .....</i>                                                         | <i>4</i>  |
| <i>Introducción .....</i>                                                                              | <i>5</i>  |
| <i>1. Tipos de arquitectura de un modelo de datos.....</i>                                             | <i>7</i>  |
| 1.1 Arquitectura centralizada .....                                                                    | 7         |
| 1.2 Arquitectura distribuida .....                                                                     | 8         |
| 1.3 Arquitectura cliente-servidor en aplicaciones web .....                                            | 8         |
| <i>2. Arquitectura SQL y NoSQL .....</i>                                                               | <i>9</i>  |
| 2.1 Bases de datos SQL .....                                                                           | 9         |
| 2.2 Bases de datos NoSQL.....                                                                          | 9         |
| 2.3 Selección de SQL o NoSQL según requerimientos técnicos .....                                       | 10        |
| <i>3. Conceptos de modelos de datos y componentes: tablas, claves, tuplas, atributos, índices.....</i> | <i>11</i> |
| 3.1 Tablas, atributos y tuplas.....                                                                    | 11        |
| 3.2 Claves primarias y claves foráneas .....                                                           | 11        |
| 3.3 Índices .....                                                                                      | 11        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <i>4. Seguridad: consultas parametrizadas, validaciones backend</i> | <i>12</i> |
| 4.1 Consultas parametrizadas                                        | 12        |
| 4.2 Validaciones <i>backend</i>                                     | 13        |
| 4.3 Seguridad en transacciones                                      | 13        |
| <i>5. Obtención de requerimientos para un modelo de datos</i>       | <i>14</i> |
| <i>6. Modelado entidad-relación</i>                                 | <i>15</i> |
| 6.1 Entidades y atributos                                           | 15        |
| 6.2 Relaciones y cardinalidad                                       | 15        |
| <i>7. Instrucciones DML – DDL – TML - RMU-RMW</i>                   | <i>16</i> |
| 7.1 Instrucciones DDL, DML y TML                                    | 16        |
| 7.2 Instrucciones RMU y RMW                                         | 17        |
| <i>Cierre</i>                                                       | <i>18</i> |
| <i>Bibliografía</i>                                                 | <i>20</i> |

# Aprendizaje esperado de la semana

Analizan arquitectura de base de datos para aplicación web, considerando seguridad en transacciones y tipos de instrucciones.



**Fuente:** Freepick, (2026)

# Introducción



¿Qué decisiones de arquitectura de bases de datos permiten que una aplicación web sea segura, eficiente y confiable cuando gestiona múltiples usuarios y transacciones simultáneas?

En el desarrollo de aplicaciones web modernas, la base de datos cumple un rol central, ya que no solo almacena información, sino que permite organizarla, protegerla y disponibilizarla de forma segura y eficiente. Cada interacción que realiza una persona usuaria —como iniciar sesión, registrar información o realizar una compra— implica una operación sobre la base de datos, lo que convierte su diseño y arquitectura en un componente crítico del sistema.

Una arquitectura de base de datos mal definida puede generar problemas de rendimiento, inconsistencias en la información o vulnerabilidades de seguridad que expongan datos sensibles. Por el contrario, una arquitectura adecuada permite soportar altos niveles de concurrencia, garantizar la integridad de los datos y proteger las transacciones frente a ataques comunes en aplicaciones web.

En esta semana se analizan los principales tipos de arquitecturas de bases de datos utilizadas en aplicaciones web, las diferencias entre modelos SQL y NoSQL, los conceptos fundamentales del modelado de datos, así como los mecanismos de seguridad y los tipos de instrucciones utilizados para operar sobre la información, siempre considerando su aplicación en contextos reales de desarrollo web.

# 1. Tipos de arquitectura de un modelo de datos

La arquitectura de un modelo de datos define la forma en que la información es organizada, almacenada y gestionada dentro de un sistema informático. En el contexto de las aplicaciones web, esta arquitectura resulta fundamental, ya que influye directamente en el rendimiento, la escalabilidad y la seguridad del sistema.

Comprender las distintas arquitecturas permite tomar decisiones técnicas informadas al momento de diseñar o evaluar una aplicación web, considerando tanto las necesidades actuales como el crecimiento futuro del sistema.

## 1.1 Arquitectura centralizada

En una arquitectura centralizada, toda la información se almacena y gestiona en una única base de datos ubicada en un servidor principal. Este enfoque facilita la administración y el control de los datos, ya que las actualizaciones, respaldos y medidas de seguridad se aplican en un solo punto.

No obstante, presenta limitaciones relevantes en aplicaciones web con alta concurrencia, ya que un fallo en el servidor central puede afectar a todo el sistema.



## 1.2 Arquitectura distribuida

La arquitectura distribuida permite que la información se encuentre almacenada en distintos servidores o nodos que trabajan de manera coordinada. Este enfoque mejora la disponibilidad del sistema y reduce el impacto de fallos.

En aplicaciones web modernas, esta arquitectura resulta especialmente útil para soportar grandes volúmenes de usuarios y solicitudes simultáneas.

## 1.3 Arquitectura cliente-servidor en aplicaciones web

En este tipo de arquitectura, el cliente se encarga de la interacción con la persona usuaria, mientras que el servidor concentra la lógica de negocio y el acceso a la base de datos. Esta separación permite aplicar controles de seguridad en el *backend* y proteger la información almacenada.

### Ejemplo aplicado:

En una aplicación de ventas online, como un sistema de *e-commerce*, el cliente (navegador) se conecta al servidor web que maneja la lógica de negocio (por ejemplo, verificar stock, calcular descuentos). Este servidor, a su vez, se conecta a la base de datos para consultar productos y registrar órdenes. Al separar las responsabilidades, se protege la base de datos y se asegura que solo el *backend* tenga acceso directo a ella.

## 2. Arquitectura SQL y NoSQL

Las arquitecturas SQL y NoSQL representan dos enfoques distintos para la gestión de datos en aplicaciones web. La elección entre ambos depende de los requerimientos técnicos, funcionales y de escalabilidad del sistema.

### 2.1 Bases de datos SQL

Las bases de datos SQL utilizan tablas estructuradas con esquemas definidos previamente. Cada tabla representa una entidad y se relaciona con otras mediante claves primarias y foráneas, lo que permite mantener la integridad referencial.

Este tipo de arquitectura es especialmente adecuada para aplicaciones web que requieren transacciones complejas y consistencia fuerte de los datos.

### 2.2 Bases de datos NoSQL

Las bases de datos NoSQL ofrecen mayor flexibilidad en la estructura de los datos, permitiendo trabajar con documentos, colecciones, grafos o pares clave-valor. Este enfoque facilita la escalabilidad horizontal y el manejo de grandes volúmenes de información.

Son ampliamente utilizadas en aplicaciones web de alto tráfico, donde el rendimiento y la disponibilidad son prioritarios.

## 2.3 Selección de SQL o NoSQL según requerimientos técnicos

La selección entre SQL y NoSQL debe considerar el tipo de datos, la necesidad de transacciones, la escalabilidad y la complejidad del sistema. En muchos proyectos web, incluso se combinan ambos enfoques para aprovechar sus ventajas.

### **Ejemplo aplicado:**

Un sistema bancario necesita garantizar la integridad de las transacciones financieras. Por eso, es ideal utilizar una base de datos SQL como PostgreSQL. En cambio, una red social que almacena miles de publicaciones por segundo podría usar NoSQL, como MongoDB, para aprovechar su flexibilidad y escalabilidad.

# 3. Conceptos de modelos de datos y componentes: tablas, claves, tuplas, atributos, índices

Un modelo de datos permite representar de manera lógica la información que maneja una aplicación web y las relaciones entre sus distintos componentes. Comprender estos conceptos es clave para diseñar bases de datos eficientes y seguras.

## 3.1 Tablas, atributos y tuplas

Las tablas representan entidades del sistema, como usuarios o productos. Los atributos describen las características de estas entidades, mientras que las tuplas corresponden a cada uno de los registros almacenados.

## 3.2 Claves primarias y claves foráneas

Las claves primarias identifican de forma única cada registro dentro de una tabla. Las claves foráneas permiten establecer relaciones entre tablas, asegurando la coherencia de la información.

## 3.3 Índices

Los índices permiten optimizar el acceso a los datos, mejorando el rendimiento de las consultas. Su uso adecuado es especialmente importante en aplicaciones web con grandes volúmenes de información.

## 4. Seguridad: consultas parametrizadas, validaciones *backend*

La seguridad en bases de datos es un aspecto crítico en el desarrollo de aplicaciones web, debido al manejo de información sensible y a la exposición constante a internet.

### 4.1 Consultas parametrizadas

Las consultas parametrizadas permiten separar el código SQL de los datos ingresados por la persona usuaria, evitando ataques de inyección SQL. Esta práctica es considerada un estándar en el desarrollo seguro de aplicaciones web.

#### **Ejemplo aplicado:**

Supongamos que una aplicación web permite iniciar sesión con usuario y contraseña. Una consulta no parametrizada podría ser vulnerable:

```
SELECT * FROM usuarios WHERE usuario = 'admin' AND clave = '1234';
```

En cambio, una consulta parametrizada en PHP sería:

```
$stmt = $conn->prepare("SELECT * FROM usuarios WHERE usuario = ?  
AND clave = ?");
```

```
$stmt->bind_param("ss", $usuario, $clave);
```

Esto evita que un atacante modifique la consulta inyectando código malicioso.

## 4.2 Validaciones *backend*

Las validaciones realizadas en el *backend* permiten asegurar que los datos recibidos cumplen con las reglas del sistema, incluso si las validaciones del lado del cliente son omitidas o manipuladas.

## 4.3 Seguridad en transacciones

El uso de transacciones permite asegurar que un conjunto de operaciones sobre la base de datos se ejecute de manera completa o no se ejecute, manteniendo la integridad de la información.

## 5. Obtención de requerimientos para un modelo de datos

La obtención de requerimientos corresponde a la etapa inicial del diseño de un modelo de datos. En esta fase se identifican las necesidades del sistema, los procesos que se deben soportar y la información que será almacenada.

Un levantamiento adecuado de requerimientos permite diseñar una base de datos coherente, escalable y alineada con los objetivos de la aplicación web.

## 6. Modelado entidad-relación

El modelado entidad-relación es una técnica que permite representar gráficamente las entidades, sus atributos y las relaciones entre ellas. Este modelo facilita la comprensión del sistema antes de su implementación física.

### 6.1 Entidades y atributos

Las entidades representan objetos relevantes del sistema, mientras que los atributos describen sus características principales.

### 6.2 Relaciones y cardinalidad

Las relaciones indican cómo interactúan las entidades entre sí, y la cardinalidad define cuántos elementos de una entidad pueden relacionarse con otra.

#### **Ejemplo aplicado:**

En un sistema de biblioteca, una relación uno a muchos se da entre la entidad "Autor" y la entidad "Libro", ya que un autor puede escribir varios libros, pero cada libro tiene un solo autor. Esta relación se representa mediante una clave foránea en la tabla de libros.



## 7. Instrucciones DML – DDL – TML – RMU-RMW

Las instrucciones de bases de datos permiten definir, manipular y controlar la información almacenada, siendo fundamentales para el funcionamiento de una aplicación web.

### 7.1 Instrucciones DDL, DML y TML

Las instrucciones DDL permiten definir la estructura de la base de datos.

Las instrucciones DML permiten insertar, modificar y eliminar datos.

Las instrucciones TML permiten gestionar transacciones y asegurar la consistencia de la información.

#### **Ejemplo aplicado:**

#### **Para crear una tabla (DDL):**

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100)  
);
```

### **Para insertar datos (DML):**

```
INSERT INTO usuarios (id, nombre, correo) VALUES (1, 'Ana',  
'ana@mail.com');
```

### **Para gestionar transacciones (TML):**

```
START TRANSACTION;
```

```
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
```

```
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
```

```
COMMIT;
```

## **7.2 Instrucciones RMU y RMW**

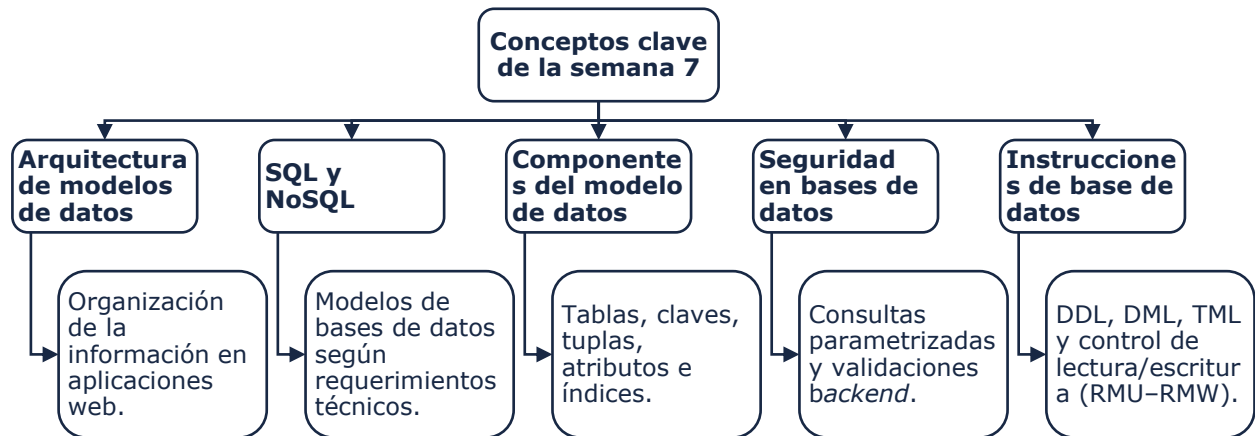
Las instrucciones RMU y RMW se utilizan en contextos específicos donde se requiere controlar la lectura y escritura concurrente de datos, especialmente en aplicaciones web con múltiples usuarios.

### **Pregunta de síntesis**

**¿Cómo influyen la arquitectura del modelo de datos, la selección entre SQL y NoSQL y el uso adecuado de instrucciones y mecanismos de seguridad en la confiabilidad de una aplicación web?**

# Cierre

Por medio del siguiente organizador gráfico se destacan las ideas clave de esta semana:



El organizador gráfico sintetiza los contenidos clave de la semana en torno a la arquitectura de bases de datos para aplicaciones web, agrupando los conceptos en cinco ejes principales que estructuran el aprendizaje de forma jerárquica y visual. Cada eje representa un componente fundamental para comprender cómo se diseñan, implementan y protegen las bases de datos en entornos web actuales.

El primer eje se centra en la arquitectura de modelos de datos, destacando la forma en que la información se organiza y gestiona dentro de una aplicación web. Este eje permite comprender la importancia de seleccionar una arquitectura adecuada para asegurar el rendimiento, la disponibilidad y la escalabilidad del sistema.

El segundo eje aborda la arquitectura SQL y NoSQL, explicando las diferencias entre ambos modelos y su selección según los requerimientos técnicos de la aplicación. Este componente permite analizar cómo el tipo de base de datos influye en la consistencia, la flexibilidad y la capacidad de crecimiento de un sistema web.

El tercer eje se enfoca en los componentes del modelo de datos, considerando elementos como tablas, claves, tuplas, atributos e índices. Este eje permite comprender cómo se estructura la información y cómo se establecen relaciones que aseguran la integridad y eficiencia en el acceso a los datos.

El cuarto eje se orienta a la seguridad en bases de datos, destacando el uso de consultas parametrizadas y validaciones Backend como mecanismos esenciales para proteger la información frente a accesos no autorizados y ataques comunes en aplicaciones web.

Finalmente, el quinto eje presenta las instrucciones de base de datos, incluyendo DDL, DML, TML y los mecanismos de control de lectura y escritura (RMU–RMW). Este eje permite comprender cómo se definen, manipulan y controlan los datos, asegurando transacciones seguras y un funcionamiento confiable del sistema.

En conjunto, este organizador gráfico permite visualizar de manera integrada cómo la arquitectura, el tipo de base de datos, la estructura de la información, la seguridad y el uso adecuado de instrucciones se articulan en el desarrollo de aplicaciones web robustas, fortaleciendo una comprensión aplicada de los desafíos técnicos presentes en el diseño de bases de datos para entornos web.

# Bibliografía

Elmasri, R., & Navathe, S. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Fowler, M. (2019). *Patterns of enterprise application architecture*. Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Rob, P., & Coronel, C. (2020). *Database systems: Design, implementation, & management* (13th ed.). Cengage Learning.

Stonebraker, M., & Hellerstein, J. M. (2005). *What goes around comes around*. ACM SIGMOD Record, 34(4), 2–8.  
<https://doi.org/10.1145/1107499.1107501>